VM Deploy Guide

VMs from a ServiceNow ticket with AAP: the flow, the settings that decide it, and ready-made configurations

Site Automation - for the deploy template's owners | site-automation 0.15.5 | October 2026

What this is. The VM deploy job on its own: the standard setup to start with, how a run flows, which ticket number to launch with, what closes the change task (and what never closes the change), where the notes go, how the VM's place is chosen, configurations to copy, where the logic lives - and every setting with its default, an example and when to set it.

> **How to read it**
> New to it: *Start here: the standard setup*, then a dry run (Check) on a test change. More than that: *Ready-made configurations*. Setting up the templates in AAP: the *Operations Runbooks Setup Guide*, VMware chapter.
> Rendered from `docs/VM_DEPLOY_GUIDE.md` and `roles/vmware_vm/defaults/main.yml` in the site-automation repository (version 0.15.5).

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

Contents

1. The VM deploy job: flow, settings, configurations 3

Start here: the standard setup . . . 3
The flow, step by step . . . 4
Which number to launch with . . . 5
Closing the task: pick one . . . 6
Where the notes go . . . 8
Where the VM goes: who wins . . . 8
Leaving things out: the defaults . . . 9
Report first, then build . . . 9
What GenAI does - and when it does nothing . . . 10
Ready-made configurations . . . 10
What the requester writes . . . 12
Test it, step by step . . . 12
When something goes wrong . . . 13
Where the logic lives . . . 15

2. Every VM deploy setting: its default, an example, when to set it 17

Deploy a VM from a ServiceNow ticket (playbooks/vm_deploy_from_ticket.yml) . . . 17
A request in plain words, without GenAI (off by default) . . . 25
GenAI reads a request written in plain words (a proof of concept, off by default) . . . 25
Pick up VM request tickets on a schedule (playbooks/vm_deploy_poll.yml) . . . 27

1. The VM deploy job: flow, settings, configurations

The VM deploy job (`playbooks/vm_deploy_from_ticket.yml`) builds VMs in vCenter from a ServiceNow ticket. It has many settings; this guide puts the ones you decide on in one place: how a run flows, which ticket number to launch with, what closes the task (and what never closes the change), where the notes go, where the VM is placed, and configurations to copy. The full detail of every part is in VMWARE.md; every setting with its default is at the end of this guide.

> **Where your values go**
> Every setting below goes in the deploy job template's **Variables** in AAP (or `playbooks/group_vars/all.yml`, which an update zip never overwrites). The names here are examples: `example.mil`, `SiteA`, `*AB01*`, `Server Team`. Put your own in AAP, never in the repository.

Start here: the standard setup

Most sites need a handful of the settings. This is the setup to begin with - and to show:

- **Launched with the change** (`CHG...`); the change may be any group's, and must be approved and in Implement.
- **Work notes on the change and on its task** - the task found under the change by its words.
- **One switch** decides whether the job closes the task when every VM is built, or your team does.
- **The change is never closed.**

The standard setup

```
vm_deploy_templates: [rhel9-gold, WIN2022-TEMPLATE]       # only these are ever built
vm_deploy_template_specs: {"WIN2022*": Win2022-Sysprep}   # Windows: its Sysprep specification
vm_deploy_defaults: {cluster: PROD-CL01, datastore: DSC-PROD, folder: Servers/New}
vm_deploy_assignment_group: ""                     # the change may be any group's
vm_deploy_notes_also: [parent, tasks]              # every note on the change and on its task
vm_deploy_notes_tasks_query: "short_descriptionLIKEto deploy"   # which task: its words
vm_deploy_close_tasks: false                       # THE SWITCH: true = the task closed when built
vm_deploy_done_fields_by_table:                    # how a task is closed (only with the switch on)
  change_task: {state: "3", close_code: successful, close_notes: "VM deployed by AAP"}
vm_deploy_done_fields_issues_by_table:             # built, but e.g. no IP address in time
  change_task: {close_code: successful_issues}
report_email_to: vmteam@example.mil
```

- **The switch:** `vm_deploy_close_tasks: false` - every note goes on the task, your team closes it. `true` - the job closes it when every VM is built, with the `change_task` values. Nothing else changes.
- **The change stays open** because nothing gives it close values: never set `vm_deploy_done_fields` (it is for every table, the change too) and never add a `change_request` entry to `vm_deploy_done_fields_by_table`.
- **The task's words:** in ServiceNow, filter the change task list (*Short description contains* ...), right-click the filter and *Copy query*; paste it as `vm_deploy_notes_tasks_query`. The job asks for the change's open tasks that match - `short_descriptionLIKEto deploy^change_request=<the change>^active=true`. `LIKE` is *contains*, in any case. No task matches: the VMs are still built, the notes go on the change.
- **Close values** are your instance's **values**, not the labels (`3` and `successful` are ServiceNow's own): right-click the task's *State* (or *Close code*) field, *Show Choice List*, the *Value* column.

- **Launch with the change's number.** Launched with the task's number, the task is the ticket itself: its `change_task` values close it, whatever the switch says.

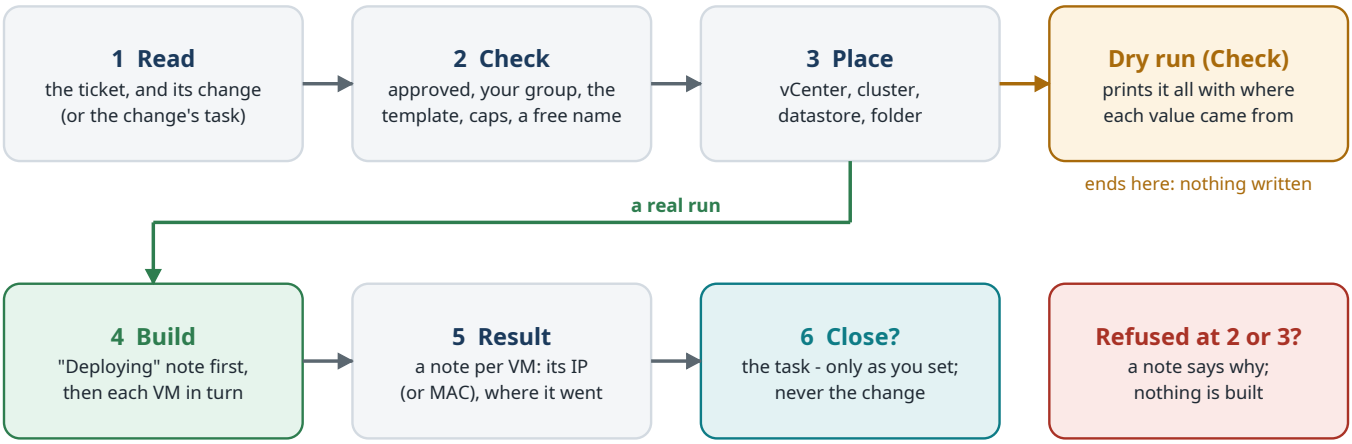
What the change and the task get, one note each step:

```
[AAP] Deploying this VM (AAP job 412 by jdoe):
vm_name: ab01web05   (from description)
template: rhel9-gold   (from description)
cluster: PROD-CL01   (from default (vm_deploy_defaults))
...
[AAP] VM ab01web05 deployed (AAP job 412 by jdoe).
IP address: 192.0.2.25
Customization: succeeded
vCenter: vc01.example.mil, datacenter DC01
Cluster: PROD-CL01, host esx01.example.mil
...
```

The job's output says it too: *"Notes on CHG0001234 and CTASK0001235. CHG0001234 is never closed (no close values for change_request); its tasks found are left open (vm_deploy_close_tasks: false)."*

What you can leave alone for now: sites and name rules (one place to build), plain words and GenAI (requests as `key: value` lines), network boot, the scheduled pickup, the approval workflow. Each is a section below; the last chapter has every setting with its default, an example, and whether it is part of this setup.

The flow, step by step



One run of the deploy job. Check (a dry run) stops after the placement; a real run writes "Deploying" before it builds anything, and closes only what you set.

1. **Read.** The job reads the ticket you launched it with - and, for a change task, its change (for a change: its open tasks of your group, for the notes). It reads `key: value` lines, plain words (`vm_deploy_plain_words`) and, when it is on, has GenAI copy the request out (`vm_deploy_genai`).
2. **Check.** The change is approved and in Implement (`vm_deploy_ready_by_table`); the ticket is your group's; the template is allowed; CPU, memory and disks are within the caps; each name is valid and free in every vCenter. Several VMs: each is also checked in vCenter (its name, template, place, customization) before the first is built. Anything wrong: a note on the ticket says what - **nothing is built**.
3. **Place.** For each VM: its vCenter, datacenter, cluster, datastore and folder (*Where the VM goes*, below). **A dry run (Check) ends here**: it prints the request, where each value came from and where each VM

would go, and writes nothing on the ticket.

4. **Build.** "[AAP] Deploying ..." on the ticket first (no note = nothing built), then each VM in turn: clone, CPU, memory, disks, customization (or network boot), power on, wait for its IP address. Each VM carries the ticket in its notes from the clone on (`[site-automation ticket=CTASK0001235 state=building]`, then `state=built`): a job that stopped is picked up where it stopped.
5. **Result.** One note per VM, written as soon as that VM is done: "[AAP] VM ab01web05 deployed" with its IP (or MAC for a network boot).
6. **Close - only what you said.** The task closes only when you set its close values and every VM is built (*Closing the task*, below). The change is never closed.

Who does what: ServiceNow holds the request and shows the notes. AAP reads, checks, writes the notes and builds - with the vCenter and ServiceNow credentials. GenAI (when on) only copies values out of the ticket text; it decides nothing, sees no credential and builds nothing. A person approves the build when the workflow has an approval step.

Which number to launch with

The deploy template's survey asks for one ticket number. **Launch with the change** (`CHG...`): the team that raises the change knows its number. A change task's number (`CTASK...`) works too, and is what the scheduled pickup uses.



The same settings work for both. Only the ticket launched - and, launched with the change, its task with vm_deploy_close_tasks - is ever closed.

	Launched with the change (CHG...)	Launched with the change task (CTASK...)
Read	the change; its open tasks of `vm_deploy_assignment_group` are found	the task, then its change (the request may be in either)
Must be approved and in Implement	the change	the task's change
Group check	the change: `vm_deploy_assignment_group` (`""` = any group), or any group with `vm_deploy_assignment_group_by_table: {change_request: ""}`	the task: `vm_deploy_assignment_group`

	Launched with the change (CHG...)	Launched with the change task (CTASK...)
Notes, with `vm_deploy_notes_also: [parent, tasks]`	the change **and** its task(s) found	the task **and** its change
Closed	the task(s), only with `vm_deploy_close_tasks: true` and a `change_task` close entry	the task, when there is a `change_task` close entry
Never closed	the change	the change

- **How the change's task is found:** its open tasks assigned to `vm_deploy_assignment_group`, and/or matching `vm_deploy_notes_tasks_query` (its words) - at least one of the two. The standard setup uses the words alone (any group); with a group set, `vm_deploy_assignment_group_by_table: {change_request: ""}` lets the change itself be any group's while its task must be yours.
- **No task matches** (not created yet, closed, other words): the job still builds, notes the change only, and says so: *"No open task of CHG... matches short_descriptionLIKEto deploy: the notes are on the change only."*
- **Several tasks match** under one change: each gets the notes (and is closed, when closing is on).
- **Which text wins** when both the change and the task describe the VMs: the task's - from the least to the most, the change, the task's short description and description, its work notes, its comments. `vm_deploy_parent_wins: true` puts the change over the task's description (a work note or comment on the task still wins). A `key: value` line and plain words rank by where they are written, not how.
- An incident or a catalog task works the same way as a task, without a change: its own table's close entry (`incident`, `sc_task`) closes it.

Closing the task: pick one

You want	Set this on the deploy template
The task closed by hand (the job never closes anything)	`vm_deploy_close_tasks: false` (the default). A `change_task` entry may stay: launched with the change, it is used only with the switch on
The task closed when every VM is built	`vm_deploy_close_tasks: true` and the `change_task` entry (below)
... closed with another close code when a VM is built but something did not finish	add `vm_deploy_done_fields_issues_by_table`
... but a network-boot (MECM) VM's task left open until someone checks the install	add `vm_deploy_done_fields_pxe: false`
The change never closed	keep `vm_deploy_done_fields: {}` (the default) and no `change_request` entry anywhere

Closed by hand - every note still goes on the task, *"[AAP] VM ab01web05 deployed"* included; your team closes it:

The task closed by hand

```
vm_deploy_notes_also: [parent, tasks]
vm_deploy_close_tasks: false
```

Closed when complete:

The task closed when every VM is built

```
vm_deploy_notes_also: [parent, tasks]
vm_deploy_close_tasks: true
vm_deploy_done_fields_by_table:
  change_task: {state: "3", close_code: successful, close_notes: "VMs deployed by AAP"}
vm_deploy_done_fields_issues_by_table:
  change_task: {close_code: successful_issues}
```

What the close settings mean:

- `vm_deploy_done_fields_by_table` - **the values** that close a ticket of a table (`change_task`, `sc_task`, `incident` ...). Its `change_task` entry closes a change task the job was launched with, on its own. These are your instance's **values**, not the labels: open a closed change task, right-click its header, *Show XML*, and copy what `<state>` and `<close_code>` hold.
- `vm_deploy_close_tasks` - **launched with the change**, close its task(s) too, with those same values. On without a `change_task` entry: nothing is closed, and the job warns *"vm_deploy_close_tasks is on, but vm_deploy_done_fields_by_table has no change_task entry"*.
- `vm_deploy_done_fields_issues_by_table` - laid **over** those values when every VM is built but something did not finish: no IPv4 address in time, no customization result, the host name not applied, a network adapter not connected, `guestinfo` not set. The task's notes say what.
- `vm_deploy_done_fields_pxe: false` - a VM from a network-boot template is powered on and still installing from MECM when the job ends: its task stays open for whoever checks the install.
- `vm_deploy_done_fields` (no table) - for **every** table without an entry, **a change too**: leave it `{}`.

Always:

- **Closed only when every VM is built.** One VM not built: the task stays open with a note per VM. Fix the cause and run the job again - the VMs already built are left as they are, one it started and did not finish is finished in place (never cloned again), the rest are built, then it closes. The work notes say which (*"Picking up where an earlier run for this ticket stopped: ..."*).
- **ServiceNow refuses the close** (a mandatory field, a business rule): the VMs are built, the job stays green, and the task and the report say *"NOT closed - close it by hand: ..."* with ServiceNow's reason. Add the missing field to the close values.
- A **closed** task is never deployed again: close a task *Unsuccessful* by hand only when you give up.

Where the notes go

Note	Where it is written	Who reads it
`[AAP] Checked - ready to build ... it would go to ...`	work note (the propose step, or a GenAI proposal waiting for a person)	the approver
`[AAP] GenAI proposal ...`	work note: each value with the ticket words it came from	the approver
`[AAP] We cannot build this VM yet: ...` - the questions	**comment** (requesters see comments, not work notes)	the requester
`[AAP] Deploying ...`	work note, before anything is built	the record
`[AAP] VM ab01web05 deployed` / `NOT deployed`	work note, one per VM	the team
`[AAP] 1 of 2 VMs built; this ticket stays open`	work note	the team

- With `vm_deploy_notes_also: [parent, tasks]`, every one of them goes on the ticket the job was launched with **and** on the other one (the change, or its task). Only the launched ticket is closed - and the change's task with `vm_deploy_close_tasks`.
- The job's output says where its notes go and what it closes: *"Notes on CHG0001234 and CTASK0001235. CHG0001234 is never closed (no close values for change_request); its tasks found are closed too (vm_deploy_close_tasks)."* A dry run (Check) says the same, and writes nothing.
- A note the other ticket does not take (the account cannot update it) is a warning, not a failure.
- The requester answers a question with a **comment** on the ticket; a later comment wins over the description. Then run the job (or workflow) again.

Where the VM goes: who wins

For each VM, each placement value (`vcenter`, `datacenter`, `cluster`, `datastore`, `folder`) comes from the first of these that gives it:

1. **The ticket itself** - a `cluster:` (or `site:`, `folder:` ...) line, or the VM's own line in a change laid out one server per line.
2. **Its site** (`vm_deploy_sites`) - named by a `site:` line, the VM's own line (*Location: North Campus*), or the ticket's **Location** field. A site's name is found in any case, with or without spaces; `vm_deploy_plain_sites` adds the other ways people write the place.
3. **Its name rule** (`vm_deploy_name_rules`) - the first `match` that fits the VM's name. A rule may name a `site:` (step 2 then applies), or `sites: [SiteA, SiteB]` when the same name is used at two places: the ticket must then say which, never a guess.
4. `vm_deploy_defaults`.
5. **Automatic placement** among the candidates you list (a site's `clusters` / `datastores`, else `vm_deploy_clusters` / `vm_deploy_datastores`): the cluster with the most memory left with a host down, the least used datastore with room under 85 %. With `vm_deploy_datastore_balance: true`, a VM nothing else places goes on the least used shared datastore of its cluster (not the template's); the VMs of one ticket spread out.

Datastores by the VM's name (`vm_deploy_datastore_rules`) come before steps 2 to 6: e.g. SQL servers on the SQL datastores, `[{match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]}]` - those VMs go on the least used of their rule's datastores, and no other VM goes there, even when it is the template's own. A ticket that names a datastore outside its rule is refused.

6. **The template's own place.**

The allow-lists (`vm_deploy_clusters`, `_datastores`, `_folders`, when set) check a value from steps 1 to 4, and automatic placement chooses only among them. A site in another vCenter: that vCenter is in `vmware_vcenters`, and the template is there under the same name. The dry run shows each value's source: `cluster: A-CL01 (from site SiteA)`.

Leaving things out: the defaults

A ticket need not say everything. `vm_deploy_defaults` fills in **any** value it leaves out - the template, CPU, memory and disks as well as the place - and what the ticket says still wins:

A ticket may say only the name

```
vm_deploy_defaults:
  template: rhel9-gold
  cpu: 2
  memory_gb: 8
  cluster: PROD-CL01
  datastore: DSC-PROD
  folder: Servers/New
```

The ticket says	What is built
`vm_name: ab01web05`	rhel9-gold, 2 vCPU, 8 GB, in PROD-CL01 - each value shown as `(from default (vm_deploy_defaults))`
`vm_name: ab01web06`, `template: WIN2022-TEMPLATE`, `cpu: 4`, `memory_gb: 16`	the ticket's template and size, in PROD-CL01

- **With sites:** `vm_deploy_defaults: {site: SiteA}` sends a ticket that names no site to SiteA; a `site: SiteB` line (or its Location) still wins. Without a default site, a ticket that names none is refused - or, with `vm_deploy_site_required: false`, placed by the rest of the defaults.
- **A default template is still checked:** it must be in `vm_deploy_templates`, and a Windows one needs its Sysprep specification in `vm_deploy_template_specs` (unless it boots from the network).
- **Nothing for the place at all:** the job chooses among `vm_deploy_clusters` / `vm_deploy_datastores` (automatic placement) - and without those, uses the template's own cluster, datastore and folder (with `vm_deploy_datastore_balance: true`, the least used datastore of that cluster).

Report first, then build

Three ways to see what would be built before anything is:

1. **Check (a dry run)** - the simplest. Launch the deploy template with *Job type* **Check** (tick *Prompt on launch* next to *Job type*). It reads the change, checks everything, chooses where each VM would go, and prints it all with where each value came from. It **writes nothing** on the ticket. Then launch it again as **Run** to build.
2. **The approval workflow** (`VM - deploy from ServiceNow ticket (workflow)`) - one launch:

- step 1 (`vm_deploy_phase: propose`) checks and places, and writes *"[AAP] Checked - ready to build after the approval … It would go to …"* on the change and its task, and emails it;
- a person clicks **Approve** in AAP (*Deny* = nothing built);
- step 3 (`vm_deploy_phase: build`) reads the change again, checks again, and builds;
- step 3 builds what the change says **then**: a change edited while the approval waited is built as edited (still within the rules) - only a GenAI proposal is tied to the ticket text it was made from;
- setting it up: VMWARE.md.

3. **GenAI without the workflow** - with `vm_deploy_genai: true`, the first run posts GenAI's proposal and stops; the next run builds that proposal.

What GenAI does - and when it does nothing

- **Off by default** (`vm_deploy_genai: false`): nothing is sent to a model. The standard setup leaves it off; tickets written as `key: value` lines - or read by the plain-words rules - never need it.
- **On**, it is asked only when the ticket's own lines and words do not give both a name and a template (or left a question). It **copies values out** of the ticket - names, the OS as written, CPU, memory, disks - each with the words it came from.
- **AAP decides everything:** it checks each quoted word is really in the ticket, picks the template from the OS words by its own rules, checks every limit, writes the proposal on the ticket, and builds. GenAI never sees a credential or vCenter, never chooses a template, a place or whether to build.
- **To show it safely:** `vm_deploy_genai_in_check: true` and a Check run - its proposal, and where the VM would go, with nothing written on the ticket.

Ready-made configurations

Each block is a complete starting point for the deploy template's Variables. Take one, change the example names to yours, run a dry run (Check) on a test ticket.

1. The smallest: `key: value` tickets, one cluster.

One cluster, tickets written as key: value lines

```
vm_deploy_templates: [rhel9-gold, WIN2022-TEMPLATE]       # REQUIRED: nothing else is deployed
vm_deploy_template_specs: {"WIN2022*": Win2022-Sysprep}   # Windows: its Sysprep specification
vm_deploy_assignment_group: Server Team
vm_deploy_defaults: {cluster: PROD-CL01, datastore: DSC-PROD, folder: Servers/New}
vm_deploy_max_cpu: 16
vm_deploy_max_memory_gb: 128
vm_deploy_domain: example.mil
report_email_to: vmteam@example.mil
```

2. Launched with the change, any group's; the team's task gets the notes and is closed by hand.

Launched with the change - the task closed by hand

```
vm_deploy_templates: [rhel9-gold, WIN2022-TEMPLATE]
vm_deploy_assignment_group: Server Team          # finds the team's task under the change
vm_deploy_assignment_group_by_table:             # the change itself may be any group's
  change_request: ""
vm_deploy_notes_also: [parent, tasks]            # every note on the change and the task
vm_deploy_close_tasks: false                     # the team closes the task
vm_deploy_plain_words: true                      # "a Windows 2022 server named ab01web05"
vm_deploy_defaults: {cluster: PROD-CL01, datastore: DSC-PROD, folder: Servers/New}
report_email_to: vmteam@example.mil
```

3. The same, the task closed when every VM is built. Configuration 2, with:

...and the task closed when complete

```
vm_deploy_close_tasks: true
vm_deploy_done_fields_by_table:
  change_task: {state: "3", close_code: successful, close_notes: "VMs deployed by AAP"}
vm_deploy_done_fields_issues_by_table:
  change_task: {close_code: successful_issues}
vm_deploy_done_fields_pxe: false     # MECM (network-boot) VMs: left open for the install check
```

4. Two sites in two vCenters; the name and the Location pick the site.

Sites, the words for them, and name rules

```
vmware_vcenters: [vc01.example.mil, vc02.example.mil]
vm_deploy_sites:
  SiteA: {vcenter: vc01.example.mil, cluster: A-CL01, datastore: A-DS01, folder: Servers/New}
  SiteB: {vcenter: vc02.example.mil, cluster: B-CL01, datastore: B-DS01, folder: Servers/New}
vm_deploy_plain_sites:              # other ways people write the place
  "north campus": SiteA
  "south campus": SiteB
vm_deploy_name_rules:
  - {match: "*AB01*", site: SiteA}            # every name with AB01 in it
  - {match: "*XY05*", sites: [SiteA, SiteB]}  # used at both: the ticket's Location says which
```

To test with one vCenter and one cluster, point both sites at it; split them later by changing only `vm_deploy_sites`.

5. Network-boot templates (MECM installs Windows).

Empty templates that boot from the network

```
vm_deploy_templates: [WIN2022-PXE, rhel9-gold]
vm_deploy_pxe_templates: ["WIN20*-PXE"]   # no customization; the MAC address goes on the ticket
vm_deploy_name_regex: '^[A-Za-z0-9-]{1,15}$'   # Windows computer names: 15 characters at most
```

6. GenAI reads plain-words requests; a person approves before the build. The workflow `VM - deploy from ServiceNow ticket (workflow)`: propose step, approval, build step (VMWARE.md).

GenAI with an approval

```
vm_deploy_plain_words: true
vm_deploy_genai: true
vm_deploy_genai_approval: true      # false = one run reads, checks and builds (a demo)
vm_deploy_genai_in_check: true      # a dry run shows GenAI's proposal too (writes nothing)
```

7. The schedule picks up the change tasks - on the pickup template (`playbooks/vm_deploy_poll.yml`), not the deploy one:

The pickup template: change tasks whose change is approved and in Implement

```
vm_deploy_assignment_group: Server Team
vm_deploy_poll_tables: [change_task]
vm_deploy_poll_prefix: "VM request"     # the tasks' short description starts with it
vm_deploy_poll_query: "change_request.approval=approved^change_request.state=-1"
vm_deploy_poll_states: ["1"]
vm_deploy_poll_dry_run: true        # the first week: list only
```

What the requester writes

In the ticket's description (or the change's), or a comment - `key: value` lines:

```
vm_name: ab01web05, ab01web06
template: rhel9-gold
cpu: 4
memory_gb: 16
disks_gb: 120, 200
site: SiteA
```

Or, with `vm_deploy_plain_words: true`, plain words:

```
Please build 2 Red Hat 9 servers named ab01web05 and ab01web06, 4 vCPU, 16 GB RAM,
storage 120GB and 200GB, Location: North Campus.
```

Or a change laid out one server per line - each line is its VM's own (size, disks, site):

```
a. spin up new 2025 VM / ab01web05 / CPU : CORE.4 / RAM : 16 GB. / HDD'S C: /120 Location: SiteA
a. spin up new 2025 VM / ab01web06 / CPU : CORE.8 / RAM : 32 GB. / HDD'S C: /120 Location: SiteB
```

- **Never a guess:** two answers for one value (`RHEL 9 or RHEL 8`), a count that does not match the names, a range (`web01 to web05`) - a question to the requester (a comment); nothing is built.
- **What wins:** the place it is written - a comment over a work note, both over the description, the ticket over its change. In the same place, a `key: value` line over the words. GenAI's proposal fills only what the ticket leaves open, then the defaults. Two answers in the place that wins are a question, never settled by GenAI or a default (GenAI may still lay out disks the words list: `120gb disk`, `50gb disk`).
- At most `vm_deploy_max_vms` (10) VMs per ticket; built one after the other.

Test it, step by step

1. **Find the names first:** run the read-only *VM - deploy settings finder* (`playbooks/vm_deploy_inventory.yml`): clusters, templates, folders and the lines to paste.
2. **A test change, not yet approved**, its description with the request, and a task under it assigned to your group. Launch the deploy template with the **change's** number as **Check**: *"is not ready to build: approval is 'Requested', not Approved"*.
3. **Approve the change, move it to Implement**, Check again: each VM, each value with *"(from ...)"*, where each would go, and *"Notes on CHG... and CTASK.... CHG... is never closed ..."* - who gets the notes, what is closed.
4. **Run it for real:** the VMs are built; the notes are on the change and the task; the task is closed or left open - as you set it.
5. **Run it again:** the VMs already built are reported, not built twice. Cancel a run of several VMs while one is building, then run again: that one is finished in place, the rest built.

When something goes wrong

You see	Do
`... is not ready to build: approval is 'Requested', not Approved; state is 'Assess', not Implement`	approve the change and move it to Implement, then run again; other labels on your instance: `vm_deploy_ready_by_table`
`REFUSED: the ticket is assigned to '...', not to ... (vm_deploy_assignment_group)`	launched with a change of another group: `vm_deploy_assignment_group_by_table: {change_request: ""}`; a task: assign it to your group
`No open task of CHG... matches ...: the notes are on the change only.`	the task is not assigned to your group, is closed, or its words do not match `vm_deploy_notes_tasks_query` - copy the query from the change task list's filter
`vm_deploy_close_tasks is on, but vm_deploy_done_fields_by_table has no change_task entry`	add the `change_task` entry (*Closing the task*) - or switch closing off
`NOT closed - close it by hand: ...`	ServiceNow's reason follows: add the mandatory field to the close values; values from *Show XML*
The close code shows in grey / not in the list	a label was sent, not a value: take the values from *Show XML*
`the ticket does not say which site`	`site: ...` in a comment, a Location the sites know (`vm_deploy_plain_sites`), a name rule - or `vm_deploy_site_required: false`
`the ticket does not say where ab01xy05 goes - SiteA or SiteB`	the name rule has `sites:`: the VM's line needs its Location
`site '...' is not one of vm_deploy_sites`	correct the ticket, or add the site
`template '...' is not in vm_deploy_templates`	allow it, or correct the ticket
`... and no template here is for it`	no allowed template for that OS version: add one, or `vm_deploy_plain_templates`
`the ticket asks for 2 VMs ... but names 1`	the requester writes every name in a comment
`a VM named ... already exists in vCenter`	a VM this ticket did not build (its notes do not carry the ticket): another name in a comment
`an earlier run of this ticket built it, and its guest customization failed`	check the VM's events in vCenter, delete it, run the job again
`was changed after the approval`	the ticket (or its change, or its Location) changed after the approval: run the workflow from the start
`... goes on a datastore ... (vm_deploy_datastore_rules)` / `datastore ... is kept for other VMs`	the ticket names a datastore its rule does not allow: leave the datastore out, or correct it
`no cluster named X in datacenter Y: X is a vCenter, not a cluster`	the source is in the output (`cluster: X (from ...)`): put a cluster's name there

You see	Do
`no cluster among ... has room` / `no datastore among ... has room`	another site or list, a smaller VM, or a higher target
`has no GenAI proposal for its text as it is now`	the ticket changed after the proposal: run the workflow from the start
`lacks the privilege ...`	the vCenter account's role (VMWARE.md, *Set it up*, step 2)
`no IPv4 address within 600 s`	DHCP on the staging VLAN, or VMware Tools in the template; the VM exists

More: VMWARE.md, Troubleshooting.

Where the logic lives

File	What is in it
`playbooks/vm_deploy_from_ticket.yml`	the playbook the template runs: it only includes the role's `deploy.yml`
`roles/vmware_vm/defaults/main.yml`	every `vm_deploy_*` setting, its default and what it does
`roles/vmware_vm/tasks/deploy.yml`	reads the ticket and its change; finds the change's task (*"the change's tasks that get the notes too"*); which step this is (propose, build); is GenAI needed
`roles/vmware_vm/tasks/deploy_genai.yml`	the GenAI step: the model copies values out, each checked against the ticket
`roles/vmware_vm/tasks/deploy_build.yml`	the rules check, every VM checked in vCenter first, the *Deploying* note (what an earlier run left), the loop over the VMs, *"close the ticket"* and *"close the change's tasks"*, the report and its email
`roles/vmware_vm/tasks/deploy_build_one.yml`	one VM: built (or left / finished, from an earlier run), then its result note at once
`roles/vmware_vm/tasks/deploy_note_also.yml`	the same note on the change, or on its task - notes only, never a close
`roles/vmware_vm/library/site_vmware_deploy.py`	the vCenter work: the ticket's mark on a VM (built, started - picked up), the template, automatic placement (least used, the datastore rules), clone, CPU, memory, disks, customization or network boot, power on, the IP address
`plugins/filter/vm_deploy_filters.py`	the decisions, in plain Python - below
`aap/extra_vars/vm_deploy_from_ticket.yml`	every setting, ready to paste into the template's Variables
`docs/vm_deploy_examples.yml`	an example for every setting, and when to set it (the last chapter)
`tests/test_vm_deploy.py`, `tests/vmware/run_vmware_test.sh`	the unit tests; whole runs against a vCenter simulator and a ServiceNow stand-in

The decisions in `plugins/filter/vm_deploy_filters.py`, by function:

Function	Decides
`vm_deploy_spec`	what the ticket asks for, and who wins: the place it is written (change < description < work notes < comments), then a GenAI proposal, then the defaults
`vm_deploy_words`	the plain words: templates, sizes, sites, one line per server

Function	Decides
`vm_deploy_check`	the rules: templates, caps, names, the group, sites and name rules, the datastore rules, approved and in Implement
`vm_deploy_checked_digest`	the ticket text the approval was for (the build step builds only that)
`vm_deploy_ready`	is the change approved and in Implement (`vm_deploy_ready_by_table`)
`vm_deploy_group_for`	which group the ticket launched must be (`vm_deploy_assignment_group[_by_table]`)
`vm_deploy_tasks_query`	the ServiceNow query for the change's tasks that get the notes
`vm_deploy_note_also`	which other tickets get each note
`vm_deploy_close_result`	did ServiceNow really close it (the values read back)
`vm_deploy_genai_task`, `vm_deploy_genai_parse`	what GenAI is asked; its answer checked word by word against the ticket
`vm_deploy_built`	the VMs an earlier run built for this ticket (never built twice)

2. Every VM deploy setting: its default, an example, when to set it

Every `vm_deploy_*` setting of the deploy job and of the scheduled pickup, in the order of `roles/vmware_vm/defaults/main.yml`: the value it has when you set nothing, the whole comment the file has for it, an example (`docs/vm_deploy_examples.yml`) and when you would set it. The other VMware settings (`vmware_vcenters`, the vCenter connection, the report email) are in the *Settings Reference* PDF.

Each one is marked:

- **Standard setup** - in *Start here: the standard setup* (chapter 1): `vm_deploy_ticket`, `vm_deploy_notes_also`, `vm_deploy_notes_tasks_query`, `vm_deploy_close_tasks`, `vm_deploy_templates`, `vm_deploy_assignment_group`, `vm_deploy_defaults`, `vm_deploy_template_specs`, `vm_deploy_done_fields_by_table`, `vm_deploy_done_fields_issues_by_table`.
- **Set when needed** - when the *When* line describes your site.
- **Leave as is** - the default is right unless you have that case.

Deploy a VM from a ServiceNow ticket (playbooks/vm_deploy_from_ticket.yml)

Setting	What it does - an example - when to set it
`vm_deploy_ticket` Default: (empty) **Standard setup**	The ticket (survey answer): INC..., TASK..., SCTASK..., RITM..., CHG... - or your instance's own prefixes (INCN..., CHGN...): the job asks ServiceNow which table a ticket is in. The ticket says what to build in 'key: value' lines - in its description, or in a comment / work note (a newer one overrides): vm_name: app01 template: rhel9-gold cpu: 4 memory_gb: 16 (cluster, datastore, folder, datacenter, vcenter, domain, customization_spec, notes: optional). docs/VMWARE.md **Example:** `vm_deploy_ticket: CHG0001234` *When:* the survey's one question - the operator types the change's number at launch; never set on the template
`vm_deploy_ticket_table` Default: (empty) **Leave as is**	leave it empty (the job finds each ticket's table); a table of your own, e.g. u_vm_request: for numbers whose prefix the job does not know (CTASK stays change_task) **Example:** `vm_deploy_ticket_table: u_vm_request` *When:* a table of your own whose number prefix the job does not know
`vm_deploy_ticket_prefixes` Default: `{}` **Leave as is**	Your instance's own number prefixes and their tables, e.g. {INCN: incident, CHGN: change_request}. Rarely needed: for a prefix it does not know, the job asks ServiceNow's task table (and takes INCN as INC, CHGN as CHG when it may not read that table). A table as people say it is understood: change / change request -> change_request, change task -> change_task, catalog task -> sc_task. **Example:** `vm_deploy_ticket_prefixes: {CHGN: change_request, INCN: incident}` *When:* your instance numbers tickets with its own prefixes and the API account may not read the task table

Setting	What it does - an example - when to set it
`vm_deploy_parent_tables` Default: `[change_request]` **Leave as is**	The request may also be on the ticket's parent - e.g. a task (SCTASK, CTASK) whose change holds the requirements: the parent's 'key: value' lines (and, with vm_deploy_plain_words, its plain words) are read too, under the ticket's own (what the ticket says wins). A parent that cannot be read, is closed, or is not ready (vm_deploy_ready_by_table) stops the build. Only a parent in one of these tables is read ([] = never read a parent): **Example:** `vm_deploy_parent_tables: [change_request, sc_req_item]` *When:* a catalog task whose requested item (RITM) holds the request - with request_item in vm_deploy_parent_fields
`vm_deploy_parent_fields` Default: `[change_request, parent]` **Leave as is**	The ticket's fields that point to its parent - the first one set is used: a change task's change_request, any ticket's parent (or your instance's own field, e.g. u_change): **Example:** `vm_deploy_parent_fields: [change_request, request_item, parent]` *When:* the field that points to the parent is another one (a catalog task's request_item, a field of your own)
`vm_deploy_parent_text_fields` Default: `[description]` **Set when needed**	The parent's fields with the request, in this order (a later one wins), e.g. [description, implementation_plan]; comments and work_notes may be listed too (oldest first). GenAI reads them as well. **Example:** `vm_deploy_parent_text_fields: [description, implementation_plan, comments]` *When:* launched with the task, and the change keeps the request in its implementation plan, or answers come as comments on it
`vm_deploy_parent_wins` Default: `false` **Set when needed**	What wins, from the least to the most: the parent (the change), the ticket's short description and description, its work notes, its comments (each oldest first) - a 'key: value' line and plain words alike: the later one wins, however each is written. So the change task wins over its change, and a comment wins over a work note. true = the parent's text wins over the ticket's short description and description instead (a change task that repeats the request, maybe wrongly); its work notes and comments still win over the parent. **Example:** `vm_deploy_parent_wins: true` *When:* launched with the task, and the task repeats the change's request (maybe wrongly) - the change's description wins
`vm_deploy_notes_also` Default: `[]` **Standard setup**	Every note and comment this job writes on the ticket goes on these too (closing never does: only the ticket itself is closed): parent = the change a task belongs to (when it was read); tasks = when the job is launched with a change, its open change tasks assigned to vm_deploy_assignment_group (and matching vm_deploy_notes_tasks_query). e.g. [parent, tasks]: the change and its task both get them, whichever the job is launched with. A note ServiceNow refuses there is a warning, not a failure. **Example:** `vm_deploy_notes_also: [parent, tasks]` *When:* every note on the change and on its task - whichever of the two numbers you launch with

Setting	What it does - an example - when to set it
`vm_deploy_notes_tasks_query` Default: (empty) **Standard setup**	An extra ServiceNow query for the change's tasks that get the notes, e.g. "short_descriptionLIKEto deploy" ("" = the group's) **Example:** `vm_deploy_notes_tasks_query: "short_descriptionLIKEto deploy"` *When:* which of the change's tasks get the notes (and are closed), by their words - copy it from the change task list's filter
`vm_deploy_notes_tasks_limit` Default: `50` **Leave as is**	the change's tasks that get the notes (and are closed), at most - more is a warning **Example:** `vm_deploy_notes_tasks_limit: 100` *When:* a change with more than 50 open tasks that match - the job warns when it reaches the limit
`vm_deploy_close_tasks` Default: `false` **Standard setup**	Launched with a change: also close its tasks found that way (with vm_deploy_done_fields_by_table's change_task entry, when every VM is built) - the change itself is not closed by this. They get the notes too. **Example:** `vm_deploy_close_tasks: true` *When:* the one switch - launched with the change, close its task when every VM is built; false = your team closes it
`vm_deploy_ready_by_table` Default: `change_request: {approval: Approved, state: Implement}` **Leave as is**	Before anything is built, a ticket of these tables - the ticket itself, or the parent it belongs to (a change task's change) - must have these fields: one value or a list (any of them), written as the ticket's form shows them (upper / lower case ignored). Checked by the dry run, the propose step AND right before the build: a change sent back while the approval waited is not built. Your instance's labels, e.g. {change_request: {approval: Approved, state: [Implement, Scheduled]}}; {} = no check. **Example:** `vm_deploy_ready_by_table: {change_request: {approval: Approved, state: [Implement, Scheduled]}}` *When:* your change states have other labels, or a scheduled change may be built too
`vm_deploy_templates` Default: `[]` **Standard setup**	What a ticket may ask for (names; * and ? allowed). Templates MUST be listed: nothing else is deployed. The others: [] = any. e.g. [rhel9-gold, win2022-gold] **Example:** `vm_deploy_templates: [rhel9-gold, WIN2022-TEMPLATE]` *When:* always - only these templates are ever deployed (* and ? allowed)
`vm_deploy_clusters` Default: `[]` **Set when needed**	clusters (and standalone hosts) a ticket may use; also the candidates of automatic placement **Example:** `vm_deploy_clusters: ["PROD-CL*"]` *When:* limit the clusters a ticket may name - and let the job choose among them when nothing names one
`vm_deploy_datastores` Default: `[]` **Set when needed**	datastores (or datastore clusters) a ticket may use; also the candidates of automatic placement **Example:** `vm_deploy_datastores: ["DSC-PROD*"]` *When:* limit the datastores (or datastore clusters) - and let the job choose among them

Setting	What it does - an example - when to set it
`vm_deploy_folders` Default: `[]` **Leave as is**	VM folders a ticket may use (paths below the datacenter's VM folder, e.g. Linux/*) **Example:** `vm_deploy_folders: ["Servers/*"]` *When:* limit the VM folders a ticket may name
`vm_deploy_max_cpu` Default: `16` **Set when needed**	the most vCPUs a ticket may ask for **Example:** `vm_deploy_max_cpu: 8` *When:* a lower cap than 16 vCPUs per VM
`vm_deploy_max_memory_gb` Default: `128` **Set when needed**	the most memory a ticket may ask for, in GB **Example:** `vm_deploy_max_memory_gb: 64` *When:* a lower cap than 128 GB per VM
`vm_deploy_max_vms` Default: `10` **Leave as is**	A ticket may ask for several VMs ("2 VMs named web01 and web02", "vm_name: web01, web02", or "Hostnames:" and one name per line): one VM each, the same size - or each its own, from a line of its own - built one after the other, the ticket closed when all are built. More names than this: refused, nothing built (the limits above are per VM). **Example:** `vm_deploy_max_vms: 4` *When:* fewer VMs per ticket than 10
`vm_deploy_name_regex` Default: `'^[A-Za-z0-9][A-Za-z0-9-]{0,62}$'` **Leave as is**	The VM names allowed (a regular expression); e.g. '^[a-z0-9-]{1,15}$' for Windows-safe names. **Example:** `vm_deploy_name_regex: '^[A-Za-z]{2}[0-9]{2}[A-Za-z0-9-]{1,11}$'` *When:* your naming standard - here two letters, two digits, then up to 11 more (15 in all, as Windows allows)
`vm_deploy_assignment_group` Default: (empty) **Standard setup**	Only tickets assigned to this group (its name as ServiceNow shows it), or to one of a list of groups (["Group A", "Group B"]); "" = any group (the pickup needs one). **Example:** `vm_deploy_assignment_group: Server Team` *When:* only tickets (and the change's tasks) of this group; "" = any group - the task then found by its words alone (the standard setup)
`vm_deploy_assignment_group_by_table` Default: `{}` **Set when needed**	The same for the ticket the job is launched with, by its table - in place of vm_deploy_assignment_group for that table; "" = any group. e.g. {change_request: ""}: launched with a change, the change may be assigned to any group (it must still be approved: vm_deploy_ready_by_table); its tasks that get the notes (vm_deploy_notes_also, vm_deploy_close_tasks) are still those of vm_deploy_assignment_group. **Example:** `vm_deploy_assignment_group_by_table: {change_request: ""}` *When:* with vm_deploy_assignment_group set - the change itself may be any group's, its task still your group's

Setting	What it does - an example - when to set it
`vm_deploy_defaults` Default: `{}` **Standard setup**	What a ticket does not say: e.g. {cluster: PROD-CL01, datastore: DS-CLUSTER-01, folder: Linux/New}. Not set = the template's own cluster, datastore and folder; its CPU and memory. **Example:** `vm_deploy_defaults: {template: rhel9-gold, cpu: 2, memory_gb: 8, cluster: PROD-CL01, datastore: DSC-PROD, folder: Servers/New}` *When:* what a ticket leaves out - template, size and place included; what the ticket says still wins
`vm_deploy_sites` Default: `{}` **Set when needed**	Sites: a ticket's 'site: A' line (or its Location field, or with plain words a site's name or its words in vm_deploy_plain_sites - each VM by its own line) picks the placement - over vm_deploy_defaults and the name rules; a cluster / datastore / folder the ticket names itself still wins. A site in another vCenter: its vcenter (in vmware_vcenters, the same account) - and the template there under the same name. vm_deploy_sites: SiteA: {cluster: A-PROD-CL01, datastore: A-DSC01, folder: Servers/New} SiteB: {cluster: B-PROD-CL01, datastore: B-DSC01, folder: Servers/New, vcenter: vc02.yoursite.mil} **Example:** `vm_deploy_sites: {SiteA: {cluster: A-CL01, datastore: A-DS01, folder: Servers/New}, SiteB: {vcenter: vc02.example.mil, cluster: B-CL01, datastore: B-DS01, folder: Servers/New}}` *When:* several places to build - the ticket's site line or its Location picks one
`vm_deploy_site_required` Default: `true` **Leave as is**	with sites set up: a ticket must say its site (false = vm_deploy_defaults); a name rule counts too **Example:** `vm_deploy_site_required: false` *When:* with sites set up - a ticket that names no site gets vm_deploy_defaults instead of a refusal
`vm_deploy_name_rules` Default: `[]` **Set when needed**	Name rules: the VM's name picks where it goes - the first rule whose 'match' fits the name (* and ?, upper / lower case alike) fills in what the ticket does not say: a cluster (and datastores, folder ...) or a site. The ticket's own lines, and a site the ticket names, still win. A name used at several sites: 'sites' - the ticket must say which (its Location), never a guess; its placement is in vm_deploy_sites. A rule's placement is never used for a VM its site puts in another vCenter. vm_deploy_name_rules: - {match: "*AB01*", cluster: CLUSTER-ONE} # every name with AB01 in it - {match: "*CD02*", cluster: CLUSTER-TWO, folder: Servers/New} - {match: "*EF03*", site: SiteB} # or a site from vm_deploy_sites - {match: "*GH04*", sites: [SiteA, SiteB]} # at either site: the ticket says which [{match: "*AB01*", cluster: ...}, ...] - see above **Example:** `vm_deploy_name_rules: [{match: "*AB01*", site: SiteA}, {match: "*XY05*", cluster: B-CL01}]` *When:* the VM's name says where it goes (a site code in the name)
`vm_deploy_pxe_templates` Default: `[]` **Set when needed**	Templates with no operating system yet, that install it themselves: they boot from the network (PXE, e.g. an MECM task sequence) or from their CD / ISO (e.g. a kickstart). No guest customization, no waiting for an IP address; the MAC address and the computer name go on the ticket. e.g. [win2022-pxe] **Example:** `vm_deploy_pxe_templates: ["WIN20*-PXE"]` *When:* empty templates that install their OS from the network (MECM) or a CD - no customization, the MAC on the ticket

Setting	What it does - an example - when to set it
`vm_deploy_guestinfo` Default: `true` **Leave as is**	Give a VM from one of those empty templates the ticket's name and number as VM settings its installer can read: guestinfo.hostname and guestinfo.ticket (a kickstart with VMware Tools in the installer: vmtoolsd --cmd "info-get guestinfo.hostname"). Needs the vCenter privilege Change configuration > Advanced configuration; without it the VM is still built (a note says so). false = none. **Example:** `vm_deploy_guestinfo: false` *When:* the vCenter account lacks Advanced configuration, and the installer reads no guestinfo anyway
`vm_deploy_template_aliases` Default: `{}` **Leave as is**	Short names for long template names - for the ticket's 'template:' line and GenAI's catalog (whose keys may be these short names): {short name: the template's name in vCenter}, e.g. {win2022: "Windows 2022 Standard Template - Site 1", rhel9: "RHEL 9 Server Template"} A ticket may also write a template's own name in other case or spacing: it is matched to the spelling in vm_deploy_templates. **Example:** `vm_deploy_template_aliases: {win2022: "Windows 2022 Standard Template - Site 1", rhel9: rhel9-gold}` *When:* long template names - tickets and GenAI write the short one
`vm_deploy_template_specs` Default: `{}` **Standard setup**	The customization specification (vCenter > Policies and Profiles) for each template, when the ticket, its site and the name rules name none: {template name or pattern: spec}. A Windows template that is not network-boot needs one (Sysprep: administrator password, domain or workgroup), e.g. {"Windows 2022*": Win2022-Sysprep-Domain}. Linux templates need none (the job's own DHCP one). **Example:** `vm_deploy_template_specs: {"WIN2022*": Win2022-Sysprep}` *When:* every Windows template that is not network-boot - its Sysprep specification (password, domain)
`vm_deploy_max_disks` Default: `5` **Leave as is**	disks a ticket may ask for, disk 1 included (disks_gb: 100, 200 = disk 1 grown to 100 GB, one 200 GB disk added) **Example:** `vm_deploy_max_disks: 3` *When:* fewer disks per VM than 5 (disk 1 included)
`vm_deploy_max_disk_gb` Default: `2048` **Leave as is**	the biggest disk a ticket may ask for, in GB **Example:** `vm_deploy_max_disk_gb: 1024` *When:* a smaller biggest disk than 2 TB
`vm_deploy_max_total_disk_gb` Default: `4096` **Leave as is**	all its disks together, in GB **Example:** `vm_deploy_max_total_disk_gb: 2048` *When:* less disk per VM in all than 4 TB
`vm_deploy_mem_target_pct` Default: `80` **Leave as is**	Automatic placement - when the ticket, its site and vm_deploy_defaults name no cluster / datastore: the job chooses among a site's 'clusters' / 'datastores' lists (e.g. SiteA: {clusters: [A-PROD-*]}), else among vm_deploy_clusters / vm_deploy_datastores; with neither, the template's own. a cluster fits when its memory, this VM included and a host down, stays under this % **Example:** `vm_deploy_mem_target_pct: 70` *When:* automatic placement keeps more memory free (a cluster fits while it stays under this, a host down)

Setting	What it does - an example - when to set it
`vm_deploy_failover_hosts` Default: `1` **Leave as is**	hosts to plan as failed when choosing a cluster **Example:** `vm_deploy_failover_hosts: 2` *When:* plan for two hosts down when choosing a cluster (large clusters)
`vm_deploy_datastore_max_used_pct` Default: `85` **Leave as is**	a datastore fits when it stays under this % with the VM at its full size **Example:** `vm_deploy_datastore_max_used_pct: 80` *When:* automatic placement keeps more datastore space free
`vm_deploy_datastore_rules` Default: `[]` **Set when needed**	Datastores by the VM's name: the first rule whose match fits the name (* and ?, any case) gives the datastores it goes on - automatic placement chooses among them on its cluster, the least used first, over a site's or vm_deploy_defaults' datastore (one the ticket names must be one of them). A VM no rule matches never goes on a datastore a rule names: they are kept for those VMs. e.g. SQL servers: - {match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]} **Example:** `vm_deploy_datastore_rules: [{match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]}]` *When:* some servers have datastores of their own (SQL servers on the SQL datastores) - and no other VM goes there
`vm_deploy_datastore_balance` Default: `false` **Set when needed**	true = a VM whose ticket, site, name rule and vm_deploy_defaults name no datastore (and no vm_deploy_datastores) goes on the least used shared datastore of its cluster - the location's - the VMs of one ticket spread out (each counts the ones before it); false = the template's own datastore. **Example:** `vm_deploy_datastore_balance: true` *When:* spread the VMs - each on the least used shared datastore of its cluster, not the template's
`vm_deploy_resume` Default: `true` **Leave as is**	Run again for the same ticket (a job that stopped: failed, cancelled, timed out), pick up where it stopped: each VM carries its ticket in its notes from the clone on - one it built is left as it is, one it started and did not finish (cloned, not yet powered on) is finished where it is; the work notes say which. false = a VM with the name is refused, as any other. **Example:** `vm_deploy_resume: false` *When:* a job that stopped must never finish a VM it started - a VM of that name is refused, delete it by hand
`vm_deploy_domain` Default: `localdomain` **Set when needed**	the DNS domain Linux customization sets (a ticket's domain: wins) **Example:** `vm_deploy_domain: example.mil` *When:* the DNS domain Linux VMs get from guest customization
`vm_deploy_customize` Default: `true` **Leave as is**	guest customization: host name = vm_name, every adapter on DHCP **Example:** `vm_deploy_customize: false` *When:* the template sets itself up (cloud-init) - vCenter sets no host name or DHCP
`vm_deploy_power_on` Default: `true` **Leave as is**	power the new VM on (false = leave it off, e.g. to check it first) **Example:** `vm_deploy_power_on: false` *When:* someone checks the VM before it is switched on

Setting	What it does - an example - when to set it
`vm_deploy_wait_customization` Default: `900` **Leave as is**	seconds to wait for vCenter's customization result **Example:** `vm_deploy_wait_customization: 1800` *When:* a slow Sysprep (Windows joining the domain)
`vm_deploy_wait_ip` Default: `600` **Leave as is**	seconds to wait for an IPv4 address (DHCP, through VMware Tools) **Example:** `vm_deploy_wait_ip: 1200` *When:* slow DHCP on the staging network
`vm_deploy_done_fields` Default: `{}` **Leave as is**	Fields set on the ticket when the VM is up, e.g. to resolve an incident: {state: "6", close_code: "Solution provided", close_notes: "VM deployed"} (your instance's values) **Example:** `vm_deploy_done_fields: {state: "6", close_code: "Solution provided", close_notes: "VM deployed"}` *When:* NEVER with changes - it closes every ticket without its own entry, a change too; for incidents prefer vm_deploy_done_fields_by_table {incident: ...}
`vm_deploy_done_fields_by_table` Default: `{}` **Standard setup**	The same for one table (the ticket's table: incident, sc_task, change_task ...), in place of vm_deploy_done_fields for it - e.g. close the task, never the change it belongs to: {sc_task: {state: "3", close_notes: "VM deployed by AAP"}} (3 = Closed Complete; your instance's values) Sent on its own, after the result note: when ServiceNow refuses it (a mandatory field, a business rule) the note stays, and the job says so on the ticket and in its report (it stays green: the VM is built). **Example:** `vm_deploy_done_fields_by_table: {change_task: {state: "3", close_code: successful, close_notes: "VM deployed by AAP"}}` *When:* how a task is closed (your instance's values) - launched with the change, used only with vm_deploy_close_tasks
`vm_deploy_done_fields_issues_by_table` Default: `{}` **Standard setup**	Over those fields when every VM is built but something did not finish: no IPv4 address in time, no customization result, the guest's host name not applied, no network at power on, guestinfo not set - e.g. {change_task: {close_code: successful_issues}} (your instance's values; {} = closed the same way). What happened is on the ticket's work notes. **Example:** `vm_deploy_done_fields_issues_by_table: {change_task: {close_code: successful_issues}}` *When:* another close code when the VMs are built but something did not finish (no IP address in time ...)
`vm_deploy_done_fields_pxe` Default: `true` **Set when needed**	Also for a VM from a network-boot template (vm_deploy_pxe_templates), which is powered on and still installing its OS from MECM when the job ends. false = that ticket stays open for whoever checks the install. **Example:** `vm_deploy_done_fields_pxe: false` *When:* a network-boot VM's task stays open until someone checks the MECM install

A request in plain words, without GenAI (off by default)

Setting	What it does - an example - when to set it
`vm_deploy_plain_words` Default: `false` **Set when needed**	true = also read the ticket's own words, by the rules below, so a requester can write "a Red Hat 9 server named ab01web05, medium" instead of 'key: value' lines. Never a guess: two different answers in the text are a question to the requester (a comment). 'key: value' lines still work and win; with vm_deploy_genai also on, GenAI is asked when the words leave the name or template - or a question - open. **Example:** `vm_deploy_plain_words: true` *When:* requesters write sentences, or one line per server, instead of 'key: value' lines
`vm_deploy_plain_templates` Default: `{}` **Set when needed**	The words for each template (any case; "windows 2022" also matches "Windows Server 2022", "server 2025" also "a new 2025 VM"). An allowed template's own name and its short names (vm_deploy_template_aliases) count too. An OS no template is for (Server 2025, only a 2022 template) is a question - never another version; to build it from a template whose name says another (an empty MECM shell): {"server 2025": <it>}. e.g. {"rhel 9": rhel9-gold, "red hat 9": rhel9-gold, "windows 2022": WIN2022-TEMPLATE} **Example:** `vm_deploy_plain_templates: {"red hat 9": rhel9-gold, "windows 2022": WIN2022-TEMPLATE, "server 2025": WIN2025-PXE}` *When:* with plain words - the words for each template
`vm_deploy_plain_sizes` Default: `{}` **Set when needed**	Size words and what they mean, e.g. {small: {cpu: 2, memory_gb: 8}, medium: {cpu: 4, memory_gb: 16}, large: {cpu: 8, memory_gb: 32}}. "4 vCPU" and "16 GB RAM" in the ticket are read too, and win in the same text. **Example:** `vm_deploy_plain_sizes: {small: {cpu: 2, memory_gb: 8}, medium: {cpu: 4, memory_gb: 16}, large: {cpu: 8, memory_gb: 32}}` *When:* with plain words - "a medium server" means a size
`vm_deploy_plain_sites` Default: `{}` **Set when needed**	The words for each site (vm_deploy_sites), as people write the place - any case, the spaces optional ("Site Two" is also "SiteTwo"). A site's own name counts too. A change that lists its servers one per line ('new VM / ab01web05 / ... Location: Site Two') puts each VM at the site its own line names; the CPU, memory and disks on each line are that VM's too. e.g. {"site two": SiteB, "north campus": SiteA} **Example:** `vm_deploy_plain_sites: {"north campus": SiteA, "south campus": SiteB}` *When:* with sites - the ways people write each place (its Location)

GenAI reads a request written in plain words (a proof of concept, off by default)

Setting	What it does - an example - when to set it
`vm_deploy_genai` Default: `false` **Leave as is**	When the ticket's own 'key: value' lines do not give vm_name and template: ACT (GenAI, evidence mode, no commands) copies out what the ticket says - the names, the operating system as written, CPU, memory, disks - each with the ticket's words it comes from. It decides and asks nothing (its own questions are notes in the report): the job finds the template from the OS words by its own rules, and asks the requester (a comment) only what it needs and cannot read. That run builds NOTHING: it posts the proposal on the ticket. The next run builds that proposal (GenAI is not asked again), while the ticket text is unchanged; the ticket's own lines still win. Needs the ACT model key credential on the template. docs/VMWARE.md **Example:** `vm_deploy_genai: true` *When:* plain-words tickets the rules cannot read - needs the ACT model key credential
`vm_deploy_genai_in_check` Default: `false` **Leave as is**	A dry run (Check) asks GenAI too (true): it shows GenAI's proposal and where the VM would go, and writes nothing on the ticket - no proposal note, no questions to the requester. false = Check only says GenAI would be asked. **Example:** `vm_deploy_genai_in_check: true` *When:* a dry run shows GenAI's proposal too (and still writes nothing)
`vm_deploy_template_catalog` Default: `{}` **Leave as is**	Optional: templates GenAI may name by key, each with a plain description (the key: the template's exact name). Not needed when the templates' names say their OS: the job finds the template from the OS the ticket names. A key for another version than the ticket asks for is dropped. e.g. {win2022-pxe: "Windows Server 2022 (installed over the network by MECM)", rhel9-gold: "Red Hat Enterprise Linux 9"} **Example:** `vm_deploy_template_catalog: {WIN2022-PXE: "Windows Server 2022 (installed over the network by MECM)"}` *When:* GenAI on, and template names that do not say their OS
`vm_deploy_genai_approval` Default: `true` **Leave as is**	true: a GenAI proposal waits for a person - the workflow's approval (or a second run) - before it is built. false: one run reads, checks and builds, when every value is traced to the ticket and within the rules (questions or a refusal still stop it). **Example:** `vm_deploy_genai_approval: false` *When:* a demo with GenAI and no person in between - one run reads, checks and builds
`vm_deploy_phase` Default: (empty) **Leave as is**	Which step this run is: "" = one job does it all; propose = the workflow's first step (read, ask GenAI if needed, check, write where it would go on the ticket - build nothing); build = the step after the approval (build what the ticket and its proposal say - never ask GenAI). **Example:** `vm_deploy_phase: propose` *When:* set by the approval workflow's nodes (propose, then build) - never on the template
`vm_deploy_genai_model_timeout` Default: `"{{ site_act_model_timeout | default(600) }}"` **Leave as is**	seconds the model may take for its answer **Example:** `vm_deploy_genai_model_timeout: 900` *When:* a slow model

Pick up VM request tickets on a schedule (playbooks/vm_deploy_poll.yml)

Setting	What it does - an example - when to set it
`vm_deploy_poll_enabled` Default: `true` **Leave as is**	Every run (an AAP schedule, e.g. every 15 minutes) asks ServiceNow for NEW VM requests - active, assigned to vm_deploy_assignment_group (required), marked by vm_deploy_poll_prefix and/or vm_deploy_poll_query - and starts the deploy workflow once per ticket, after writing "[AAP] Picked up ..." on it. A ticket with any note of AAP's on it is never picked up again. Nothing is built by this job: the workflow reads, checks, asks for the approval and builds. Needs the "ServiceNow API" credential and a "Red Hat Ansible Automation Platform" credential (an AAP user that may only start that workflow). docs/VMWARE.md, "Pick up tickets automatically" false = the job runs, picks up nothing and says so (the off switch) **Example:** `vm_deploy_poll_enabled: false` *When:* the pickup's off switch - the schedule runs and picks up nothing
`vm_deploy_poll_tables` Default: `[incident]` **Set when needed**	the ServiceNow tables to look in: incident, sc_task, task ... **Example:** `vm_deploy_poll_tables: [change_task]` *When:* pick up change tasks rather than incidents
`vm_deploy_poll_prefix` Default: `"VM request"` **Set when needed**	a VM request's short description starts with this ("" = mark them by vm_deploy_poll_query) **Example:** `vm_deploy_poll_prefix: ""` *When:* the tickets are found by vm_deploy_poll_query, not by how their short description starts
`vm_deploy_poll_query` Default: (empty) **Set when needed**	An extra ServiceNow encoded query, ANDed with the rest, e.g. "category=vm_request" or "cmdb_ci.name=VM Request" ("" = none). ^NQ (a second query) is refused. **Example:** `vm_deploy_poll_query: "change_request.approval=approved^change_request.state=-1"` *When:* change tasks whose change is approved and in Implement (copy your own from the list's filter)
`vm_deploy_poll_states` Default: `["1"]` **Leave as is**	the ticket states to pick up (state values: "1" = New) **Example:** `vm_deploy_poll_states: ["1", "-5"]` *When:* pick up tasks in other states too (your instance's values; -5 = Pending out of the box)
`vm_deploy_poll_max` Default: `3` **Leave as is**	the most tickets started per run; the rest wait for the next run **Example:** `vm_deploy_poll_max: 5` *When:* more tickets started per run
`vm_deploy_poll_scan` Default: `100` **Leave as is**	the most tickets read per table and run, oldest first (tickets AAP already has count too) **Example:** `vm_deploy_poll_scan: 200` *When:* many tickets in the queue
`vm_deploy_poll_workflow` Default: `"VM - deploy from ServiceNow ticket (workflow)"` **Leave as is**	the workflow job template started per ticket **Example:** `vm_deploy_poll_workflow: "VM - deploy (workflow, test)"` *When:* your deploy workflow has another name

Setting	What it does - an example - when to set it
`vm_deploy_poll_dry_run` Default: `false` **Set when needed**	true = only list what it would pick up: no note, no workflow (Check does the same) **Example:** `vm_deploy_poll_dry_run: true` *When:* the first week - only list what it would pick up
`vm_deploy_poll_claim_fields` Default: `{}` **Set when needed**	Fields set on the ticket together with the "Picked up" note, e.g. {state: "2"} (In Progress) - so the ticket leaves the states picked up as well ({} = only the note). **Example:** `vm_deploy_poll_claim_fields: {state: "2"}` *When:* move a ticket picked up to In Progress (your instance's value)
`vm_deploy_poll_aap_ca_path` Default: (empty) **Leave as is**	a CA file for AAP's own HTTPS certificate ("" = what the execution environment trusts) **Example:** `vm_deploy_poll_aap_ca_path: /etc/pki/ca-trust/source/anchors/site-ca.pem` *When:* AAP's HTTPS certificate is from your own CA, which the execution environment does not trust